Glossary

NOTE

The definitions in this glossary are short and simple, intended to convey the core idea but not the full subtleties of a term. For more detail, follow the references to the main text.

---

asynchronous

Not waiting for something to complete (e.g., sending data over the network to another node), and not making any assumptions about how long it is going to take. See "Synchronous Versus Asynchronous Replication", "Synchronous Versus Asynchronous Networks", and "System Model and Reality".

atomic

1. In the context of concurrency, describes an operation that appears to take effect at a single point in time, so another concurrent process can never encounter the operation in a "half-finished" state. See also *isolation*.
2. In the context of transactions, describes grouping together a set of writes that must either all be committed or all be rolled back, even if faults occur. See "Atomicity" and "Two-Phase Commit".

backpressure

Forcing the sender of data to slow down when the recipient cannot keep up with it. Also known as *flow control*. See "When an Overloaded System Won't Recover".

batch process

A computation that takes a fixed (and usually large) set of data as input and produces other data as output, without modifying the input. See Chapter 11.

bounded

Having a known upper limit or size. Used, for example, in the context of network delay (see "Timeouts and Unbounded Delays") and datasets (see the introduction to Chapter 12).

Byzantine fault

A fault that is characterized by a node behaving incorrectly in an arbitrary way (e.g., by sending contradictory or malicious messages to other nodes). See "Byzantine Faults".

cache

A component that remembers recently used data in order to speed up future reads of the same data. It is generally not complete; any data that is missing from the cache has to be fetched from an underlying, slower data storage system that has a complete copy of the data.

CAP theorem

A widely misunderstood theoretical result that is not useful in practice. See "The CAP theorem".

causality

The dependency between events that arises when one thing "happens before" another thing in a system—for example, a later event that occurs in response to, builds upon, or should be understood in the light of an earlier event. See "The happens-before relation and concurrency".

consensus

A fundamental problem in distributed computing concerning getting several nodes to agree on something (e.g., which node should be the leader for a database cluster). The problem is much harder than it seems at first glance. See "Consensus".

data warehouse

A database in which data from several OLTP systems has been combined and prepared to be used for analytics purposes. See "Data Warehousing".

declarative

Describes the properties that something should have, but not the exact steps for how to achieve it. In the context of database queries, a query optimizer takes a

declarative query and decides how it should best be executed. See “Terminology: Declarative Query Languages”.

denormalize

To introduce some amount of redundancy or duplication in a *normalized* dataset, typically in the form of a *cache* or *index*, in order to speed up reads. A denormalized value is a kind of precomputed query result, similar to a materialized view. See “Normalization, Denormalization, and Joins”.

derived data

A dataset created from other data through a repeatable process, which you could run again if necessary. Usually, derived data is needed to speed up a particular kind of read access to the data. Indexes, caches, and materialized views are examples of derived data. See “Systems of Record and Derived Data”.

deterministic

Describes a function that always produces the same output if you give it the same input. This means it cannot depend on random numbers, the time of day, network communication, or other unpredictable things. See “The Power of Determinism”.

distributed

Running on several nodes connected by a network. Characterized by *partial failures*: a part of the system may be broken while other parts are still working, and it is often impossible for the software to know what exactly is broken. See “Faults and Partial Failures”.

durable

Storing data in such a way that you believe it will not be lost, even if various faults occur. See “Durability”.

ETL

Extract–transform–load. The process of extracting data from a source database, transforming it into a form that is more suitable for analytical queries, and loading it into a data warehouse or batch processing system. See “Data Warehousing”.

failover

In systems that have a single leader, failover is the process of moving the leadership role from one node to another. See “Handling Node Outages”.

fault-tolerant

Able to recover automatically if something goes wrong (e.g., if a machine crashes or a network link fails). See “Reliability and Fault Tolerance”.

flow control

See *backpressure*.

follower

A replica that does not directly accept any writes from clients, but only processes data changes that it receives from a leader. Also known as a *secondary*, *read replica*, or *hot standby*. See “Single-Leader Replication”.

full-text search

Searching text by arbitrary keywords, often with additional features such as matching similarly spelled words or synonyms. A full-text index is a kind of *secondary index* that supports such queries. See “Full-Text Search”.

graph

A data structure consisting of *vertices* (things that you can refer to, also known as *nodes* or *entities*) and *edges* (connections from one vertex to another, also known as *relationships* or *arcs*). See “Graph-Like Data Models”.

hash

A function that turns an input into a random-looking number. The same input always returns the same number as output. Two different inputs will likely have two different numbers as output, although two different inputs could produce the same output (this is called a *collision*). See “Sharding by Hash of Key”.

idempotent

Describes an operation that can be safely retried; if it is executed more than once, it has the same effect as if it were executed only once. See “Idempotence”.

index

A data structure that lets you efficiently search for all records that have a particular value in a particular field. See “Storage and Indexing for OLTP”.

isolation

In the context of transactions, describes the degree to which concurrently executing transactions can interfere with each other. *Serializable* isolation provides the strongest guarantees, but weaker isolation levels are also used. See "Isolation".

join

To bring together records that have something in common. Most commonly used when one record has a reference to another (a foreign key, a document reference, an edge in a graph) and a query needs to get the record that the reference points to. See "Normalization, Denormalization, and Joins" and "Joins and Grouping".

leader

When data or a service is replicated across several nodes, the leader is the designated replica that is allowed to make changes. A leader may be elected through a protocol or manually chosen by an administrator. Also known as the *primary* or *source*. See "Single-Leader Replication".

linearizable

Behaving as if only a single copy of data is in the system, which is updated by atomic operations. See "Linearizability".

locality

A performance optimization: putting several pieces of data in the same place if they are frequently needed at the same time. See "Data locality for reads and writes".

lock

A mechanism to ensure that only one thread, node, or transaction can access something, and anyone else who wants to access the same thing must wait until the lock is released. See "Two-Phase Locking" and "Distributed Locks and Leases".

log

An append-only file for storing data. A *write-ahead log* is used to make a storage engine resilient against crashes (see "Making B-trees reliable"), a *log-structured* storage engine uses logs as its primary storage format (see "Log-Structured Storage"), a *replication log* is used to copy writes from a leader to followers (see "Single-Leader Replication"), and an *event log* can represent a data stream (see "Log-Based Message Brokers").

materialize

To perform a computation eagerly and write out its result, as opposed to calculating it on demand when requested. See "Event Sourcing and CQRS".

node

An instance of software running on a computer, which communicates with other nodes via a network in order to accomplish a task.

normalized

Structured in such a way that there is no redundancy or duplication. In a normalized database, when a piece of data changes, you need to change it in only one place, not many copies in many places. See "Normalization, Denormalization, and Joins".

OLAP

Online analytical processing. An access pattern characterized by aggregating (e.g., count, sum, average) over a large number of records. See "Operational Versus Analytical Systems".

OLTP

Online transaction processing. An access pattern characterized by fast queries that read or write a small number of records, usually indexed by key. See "Operational Versus Analytical Systems".

percentile

A way of measuring the distribution of values by counting how many values are above or below a certain threshold. For example, the 95th percentile response time during some period is the time *t* such that 95% of requests in that period complete in less than *t*, and 5% take longer than *t*. See "Describing Performance".

primary key

A value (typically a number or a string) that uniquely identifies a record. In many applications, primary keys are generated by the system when a record is created (e.g., sequentially or randomly); they are not usually set by users. See also *secondary index*.

quorum

The minimum number of nodes that need to vote on an operation before it can be considered successful. See “Using quorums for reading and writing”.

rebalance

To move data or services from one node to another in order to spread the load fairly. See “Sharding of Key-Value Data”.

replication

Keeping a copy of the same data on several nodes (*replicas*) so that it remains accessible if a node becomes unreachable. See Chapter 6.

schema

A description of the structure of data, including its fields and datatypes. Whether data conforms to a schema can be checked at various points in the data’s lifetime (see “Schema flexibility in the document model”), and a schema can change over time (see Chapter 5).

secondary index

An additional data structure that is maintained alongside the primary data storage and that allows you to efficiently search for records that match a certain kind of condition. See “Multicolumn and Secondary Indexes” and “Sharding and Secondary Indexes”.

serializable

An *isolation* guarantee that if several transactions execute concurrently, they behave the same as if they had executed one at a time, in a serial order. See “Serializability”.

sharding

Splitting a large dataset or computation that is too big for a single machine into smaller parts and spreading them across several machines. Also known as *partitioning*. See Chapter 7.

shared-nothing

An architecture in which independent nodes—each with its own CPUs, memory, and disks—are connected via a conventional network, in contrast to shared-memory or shared-disk architectures. See “Shared-Memory, Shared-Disk, and Shared-Nothing Architectures”.

skew

1. Imbalanced load across shards, such that some shards have lots of requests or data and others have much less. Also known as *hot spots*. See "Skewed Workloads and Relieving Hot Spots".
2. A timing anomaly that causes events to appear in an unexpected, nonsequential order. See the discussions of *read skew* in "Snapshot Isolation and Repeatable Read", *write skew* in "Write Skew and Phantoms", and *clock skew* in "Timestamps for ordering events".

split brain

A scenario in which two nodes simultaneously believe themselves to be the leader, which may cause system guarantees to be violated. See "Handling Node Outages" and "The Majority Rules".

stored procedure

A way of encoding the logic of a transaction such that it can be entirely executed on a database server, without communicating back and forth with a client during the transaction. See "Actual Serial Execution".

stream process

A continually running computation that consumes a never-ending stream of events as input and derives output from it. See Chapter 12.

synchronous

The opposite of *asynchronous*.

system of record

A system that holds the primary, authoritative version of data, also known as the *source of truth*. Changes are first written here, and other datasets may be derived from the system of record. See "Systems of Record and Derived Data".

timeout

One of the simplest ways of detecting a fault—namely, by observing the lack of a response within a certain amount of time. However, it is impossible to know whether a timeout is due to a problem with the remote node or an issue in the network. See "Timeouts and Unbounded Delays".

total order

A way of comparing two things (e.g., timestamps) that allows you to always say which one is greater and which one is lesser. An ordering in which some things are incomparable (you cannot say which is greater or smaller) is called a *partial order*.

transaction

Grouping together several reads and writes into a logical unit in order to simplify error handling and concurrency issues. See Chapter 8.

two-phase commit (2PC)

An algorithm to ensure that several database nodes either all *atomically* commit or all abort a transaction. See "Two-Phase Commit".

two-phase locking (2PL)

An algorithm for achieving *serializable isolation* that works by a transaction acquiring a lock on all data it reads or writes and holding the lock until the end of the transaction. See "Two-Phase Locking".

unbounded

Not having any known upper limit or size. The opposite of *bounded*.